Container Management Shell

Review 2 Presentation

25CS2104E — Operating Systems and Systems Programming

Instructor: Dr. P. Kalpana

- Vidya Keerthi Chitturi — 2510030372
- Badam Hasika — 2510030403
- Shaik Asha Parveen — 2510030225
- Madi Amulya — 2510030158

Made with GAMMA

The project brings core shell mechanisms into one learning environment

01 Existing Linux shells provide command execution and process management.

02 Understanding their internal mechanisms requires multiple operating-systems concepts.

03 This project implements selected shell mechanisms in C to demonstrate those concepts practically.

04 **Objective:** Integrate process management, IPC, redirection, signals, memory validation, and threading in one modular shell.

Made with GAMMA

Literature spans shell foundations, behavior, IPC, semantics, and practice

01 **Bourne (1978):** UNIX shell foundations.

02 **Kidwai et al. (2021):** Comparative review of Linux shells.

03 **Performance Analysis of Five IPC Mechanisms (2003):** IPC mechanism performance.

04 **Greenberg and Blatt (2019):** Formal semantics of the POSIX shell.

05 **University of Chicago Unix Shell Project:** Practical shell implementation and learning.

Made with GAMMA

The educational gap is integration, not a lack of mature shells

EXISTING CONTEXT

Existing shells: Mature, feature-rich command-line environments.

Shell literature: Focuses on shell design, behavior, comparisons, or individual OS mechanisms.

PROJECT-LEVEL MOTIVATION

Project-level motivation: A compact educational implementation can help learners observe how selected OS concepts interact.

This is an identified educational motivation, not a proven gap in all existing research.

Made with GAMMA

The proposed shell integrates selected mechanisms in modular C code

01

Dynamic input and command parsing.

02

Process creation and external command execution.

03

Built-in commands and signal handling.

04

Two-command pipelines and I/O redirection.

05

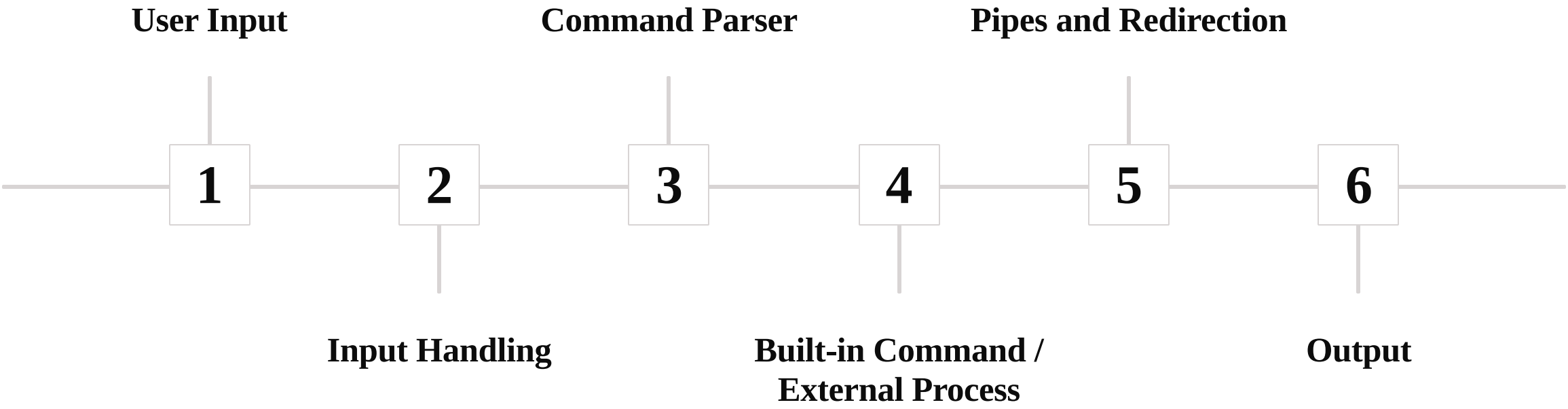
Memory-debugging tools and a detached monitoring thread.

06

Novelty: Educational integration and demonstration in one project, not newness of the individual mechanisms.

Made with GAMMA

Input moves through parsing, execution, IPC, and output



Supporting modules: signal handling; detached monitoring thread; memory debugging and validation.

Implementation: C, GCC, Linux/POSIX APIs, Makefile, Git, and GitHub.

Made with GAMMA

Implemented features demonstrate core shell and POSIX mechanisms

Scope note: no arbitrary-length pipelines and no actual container creation.

Feature	Implementation / example
Built-ins	help, pwd, cd, env, exit
External execution	fork(), execvp(), waitpid()
Two-command pipeline	pipe() and dup2()
I/O redirection	Input, output, append, and error redirection
Monitoring	Periodic status messages from a detached thread

Made with GAMMA

Validation confirms the implemented scope and surfaces current limits

VALIDATION

- Tested built-in and external commands.
- Demonstrated two-command pipelines and I/O redirection.
- GDB used for execution inspection and debugging.
- AddressSanitizer test completed without visible runtime memory-error reports.
- Valgrind reported **272 bytes possibly lost and one error context**, with zero bytes definitely lost.

Current limitations

- Only two-command pipelines.
- Monitoring thread displays status messages; it does not yet provide actual CPU/memory monitoring.
- No full container lifecycle or isolation functionality.

Future work

Resource monitoring, command history, multi-stage pipelines, background execution, and job control.

Made with GAMMA

The shell is a foundation for deeper systems programming

Conclusion

- Demonstrates practical use of Linux/POSIX system programming.
- Integrates multiple OS concepts in a modular C application.
- Provides a foundation for further shell and process-management features.

References

1. S. R. Bourne, “UNIX Time-Sharing System: The UNIX Shell,” Bell System Technical Journal, 1978. DOI: 10.1002/j.1538-7305.1978.tb02139.x.
2. A. Kidwai et al., “A Comparative Study on Shells in Linux: A Review,” Materials Today: Proceedings, 2021. DOI: 10.1016/j.matpr.2020.08.508.
3. “Performance Analysis of Five Interprocess Communication Mechanisms Across UNIX Operating Systems,” Journal of Systems and Software, 2003. DOI: 10.1016/S0164-1212(02)00134-6.
4. M. Greenberg and A. J. Blatt, “Executable Formal Semantics for the POSIX Shell,” arXiv:1907.05308, 2019. https://arxiv.org/abs/1907.05308
5. University of Chicago, “Project 4: The Unix Shell,” Operating Systems Course Project. https://www.classes.cs.uchicago.edu/archive/2019/fall/15400-1/proj/p4/p4shell.html

Made with GAMMA